

Setting up and using Meld

Asked 10 years, 9 months ago Modified 1 month ago



Sign in to Stack Exchange with Google



Maxie Schmidt

maxieds@gmail.com



Maxie Schmidt

maxieds85v2@gmail.com



431



[Meld](#) is a popular free, open-source, and tool.

- How do I set up and use Meld as my Git difftool?
- How do I set up and use Meld as my Git mergetool?

git meld

Share Improve this question Follow

edited Aug 4, 2024 at 17:56



Dr. Gut

3,912 15 33

asked Dec 6, 2015 at 16:35



mattst

14.3k 4 36 44

1 Related post - How to [configure a diff tool in Git in general](#). – RBT Apr 5, 2018 at 23:48

wiki.gnome.org/Apps/Meld/Windows: "Version control browsing on Windows is relatively untested; it may or may not work or not depending on which source control software you use" – PythoNic Nov 30, 2022 at 19:05

13 Answers

Sorted by: Highest score (default)



How do I set up and use Meld as my Git difftool?

677

[git difftool](#) displays the diff using a GUI diff program (i.e., Meld) instead of displaying the diff output in your terminal.



Although you can set the GUI program on the command line using `-t <tool> / --tool=<tool>` it makes more sense to configure it in your `.gitconfig` file. (Note: See the sections about escaping quotes and Windows paths at the bottom.)



```
# Add the following to your .gitconfig file.
[diff]
    tool = meld
[difftool]
    prompt = false
[difftool "meld"]
    cmd = meld "$LOCAL" "$REMOTE"
```

(Note: These settings will not alter the behaviour of `git diff` which will continue to function as usual.)

You use `git difftool` in exactly the same way as you use `git diff`. E.g.,

```
git difftool <COMMIT_HASH> file_name
git difftool <BRANCH_NAME> file_name
git difftool <COMMIT_HASH_1> <COMMIT_HASH_2> file_name
```

If properly configured, a Meld window will open displaying the diff using a GUI interface.

The order of the Meld GUI window panes can be controlled by the order of `$LOCAL` and `$REMOTE` in `cmd`, that is to say which file is shown in the left pane and which in the right pane. If you want them the other way around simply swap them around like this:

```
cmd = meld "$REMOTE" "$LOCAL"
```

Finally, the `prompt = false` line simply stops Git from prompting you as to whether you want to launch Meld or not, by default Git will issue a prompt.

How do I set up and use Meld as my Git mergetool?

[git mergetool](#) allows you to use a GUI merge program (i.e., Meld) to resolve the merge conflicts that have occurred during a merge.

Like `difftool`, you can set the GUI program on the command line using `-t <tool> / --tool=<tool>` but, as before, it makes more sense to configure it in your `.gitconfig` file. (Note: See the sections about escaping quotes and Windows paths at the bottom.)

```
# Add the following to your .gitconfig file.
[merge]
    tool = meld
[mergetool "meld"]
    # Choose one of these two lines (not both!) explained below.
    cmd = meld "$LOCAL" "$MERGED" "$REMOTE" --output "$MERGED"
    cmd = meld "$LOCAL" "$BASE" "$REMOTE" --output "$MERGED"
```

You do *not* use `git mergetool` to perform an actual merge. Before using `git mergetool` you perform a merge in the usual way with Git. E.g.,

```
git checkout master
git merge branch_name
```

If there is a merge conflict, Git will display something like this:

```
git merge branch_name
```

Output:

```
Auto-merging file_name
CONFLICT (content): Merge conflict in file_name
Automatic merge failed; fix conflicts and then commit the result.
```

At this point `file_name` will contain the partially merged file with the merge conflict information (that's the file with all the `>>>>>>` and `<<<<<<` entries in it).

Mergetool can now be used to resolve the merge conflicts. You start it very easily with:

```
git mergetool
```

If properly configured, a Meld window will open displaying three files. Each file will be contained in a separate pane of its GUI interface.

In the example `.gitconfig` entry above, two lines are suggested as the `[mergetool "meld"] cmd` line. In fact there are all kinds of ways for advanced users to configure the `cmd` line, but that is beyond the scope of this answer.

This answer has two alternative `cmd` lines which, between them, will cater for most users, and will be a good starting point for advanced users who wish to take the tool to the next level of complexity.

Firstly here is what the parameters mean:

- `$LOCAL` is the file in the current branch (e.g., `master`).
- `$REMOTE` is the file in the branch being merged (e.g., `branch_name`).
- `$MERGED` is the partially merged file with the merge conflict information in it.
- `$BASE` is the shared commit ancestor of `$LOCAL` and `$REMOTE`, this is to say the file as it was when the branch containing `$REMOTE` was originally created.

I suggest you use either:

```
[mergetool "meld"]
  cmd = meld "$LOCAL" "$MERGED" "$REMOTE" --output "$MERGED"
```

or:

```
[mergetool "meld"]
  cmd = meld "$LOCAL" "$BASE" "$REMOTE" --output "$MERGED"
```

See 'Note On Output File' which explains `--output "$MERGED"`.

The choice is whether to use `$MERGED` or `$BASE` in between `$LOCAL` and `$REMOTE`.

Either way Meld will display three panes with `$LOCAL` and `$REMOTE` in the left and right panes and either `$MERGED` or `$BASE` in the middle pane.

In *both* cases the middle pane is the file that you should edit to resolve the merge conflicts. The difference is just in which starting edit position you'd prefer; `$MERGED` for the file which contains the partially merged file with the merge conflict information or `$BASE` for the shared commit ancestor of `$LOCAL` and `$REMOTE`. [Since both `cmd` lines can be useful I keep them both in my `.gitconfig` file. Most of the time I use the `$MERGED` line and the `$BASE` line is commented out, but the commenting out can be swapped over if I want to use the `$BASE` line instead.]

Note On Output File: Do not worry that `--output "$MERGED"` is used in `cmd` regardless of whether `$MERGED` or `$BASE` was used earlier in the `cmd` line. The `--output` option simply tells Meld what filename Git wants the conflict resolution file to be saved in. Meld will save your conflict edits in that file regardless of whether you use `$MERGED` or `$BASE` as your starting edit point.

After editing the middle pane to resolve the merge conflicts, just save the file and close the Meld window. Git will do the update automatically and the file in the current branch (e.g. master) will now contain whatever you ended up with in the middle pane.

Git will have made a backup of the partially merged file with the merge conflict information in it by appending `.orig` to the original filename. e.g. `file_name.orig`. After checking that you are happy with the merge and running any tests you may wish to do, the `.orig` file can be deleted.

At this point you can now do a commit to commit the changes.

If, while you are editing the merge conflicts in Meld, you wish to abandon the use of Meld, then quit Meld without saving the merge resolution file in the middle pane. Git will respond with the message `file_name seems unchanged` and then ask `Was the merge successful? [y/n]`, if you answer `n` then the merge conflict resolution will be aborted and the file will remain unchanged. Note that if you have saved the file in Meld at any point then you will not receive the warning and prompt from Git. [Of course you can just delete the file and replace it with the backup `.orig` file that Git made for you.]

If you have more than one file with merge conflicts then Git will open a new Meld window for each, one after another until they are all done. They won't all be opened at the same time, but when you finish editing the conflicts in one, and close Meld, Git will then open the next one, and so on until all the merge conflicts have been resolved.

It would be sensible to create a dummy project to test the use of `git mergetool` before using it on a *live* project. Be sure to use a filename containing a space in your test, in case your OS

requires you to escape the quotes in the `cmd` line, see below.

Escaping quote characters

Some operating systems may need to have the quotes in `cmd` escaped. Less experienced users should remember that configuration command lines should be tested with filenames that include spaces, and if the `cmd` lines don't work with the filenames that include spaces then try escaping the quotes. E.g.,

```
cmd = meld \"$LOCAL\" \" $REMOTE\"
```

In some cases more complex quote escaping may be needed. The 1st of the Windows path links below contains an example of triple-escaping each quote. It's a bore, but it is sometimes necessary. E.g.,

```
cmd = meld \\\"$LOCAL\\\" \\\" $REMOTE\\\"
```

Windows paths

Windows users will probably need extra configuration added to the Meld `cmd` lines. They may need to use the full path to `meldc`, which is designed to be called on Windows from the command line, or they may need or want to use a wrapper. They should read the Stack Overflow pages linked below which are about setting the correct Meld `cmd` line for Windows. Since I am a Linux user, I am unable to test the various Windows `cmd` lines and don't have any further information on the subject other than to recommend using my examples with the addition of a full path to Meld or `meldc`, or adding the Meld program folder to your `path`.

- [Git Diff and Meld on Windows](#)
- [How to set Meld as git mergetool](#)
- [Git mergetool with Meld on Windows](#)

Share Improve this answer Follow

edited Aug 4, 2024 at 17:28



Dr. Gut

3,912 15 33

answered Dec 6, 2015 at 16:35



mattst

14.3k 4 36 44

15 Comments ^

Add a comment



KrisWebDev Over a year ago

Thanks, that's so much easier to use `[mergetool "meld"] cmd = meld "$LOCAL" "$MERGED" "$REMOTE" --output "$MERGED"` in `~/.gitconfig`, then just resolve the conflicts highlighted in red in the middle pan and save! It should be the default setting.

▲ 4 Reply ...



mattst Over a year ago

\$LOCAL \$MERGED \$REMOTE is the setting I use most of the time, when there are just a few conflicts to resolve it's excellent and it's my default as well. \$LOCAL \$BASE \$REMOTE really comes into its own when there's lots to do and you know exactly which sections of code are coming from which file; the shared commit ancestor can be a great clutter free starting point, sometimes the conflict highlighting actually gets in the way and a cleaner base is a blessing.

▲ 5 Reply ...



Alteisen Over a year ago

Note: If you are on OSX and installed meld via homebrew, the output parameter will require an = like this: `cmd = meld "$LOCAL" "$MERGED" "$REMOTE" --output="$MERGED"`

▲ 5 Reply ...



KC Baltz Over a year ago

For Mac users who installed the .dmg and find that 'meld' isn't in their path, be sure to follow the alternate set of instructions here: yousseb.github.io/meld

▲ 3 Reply ...



ChrisG Over a year ago

That is so much better than the explanation at [Git Configuration - git mergetool](#). Thank you very much especially for explaining the difference between \$MERGED and \$BASE. Saved me from going crazy!

▲ 4 Reply ...



While [mattst's answer](#) is correct, here's the fastest way to just go ahead and configure Meld as your visual diff tool. Just copy/paste this:

126



```
git config --global diff.tool meld
git config --global difftool.prompt false
```



Now run `git difftool` in a directory and Meld will be launched for each different file.



Side note: [Meld is surprisingly slow](#) at comparing CSV files, and no Linux diff tool I've found is faster than this Windows tool called [Compare It!](#) (last updated in 2010).

Share Improve this answer Follow

edited Mar 5, 2024 at 7:22



Peter Mortensen

30.8k 22 112 135

answered Oct 7, 2016 at 20:13



Dan Dascalescu

154k 66 337 424

2 Comments ^

Add a comment



BeeOnRope Over a year ago

You probably want a `git config --global difftool.meld.cmd 'meld "$LOCAL" "$REMOTE"'` line in there too. This is the "default", but as soon as you configure a `mergetool`, the `difftool` will start using the `mergetool` configuration as the default if the `diff` config isn't found. Since merge is generally configured to pass three files for a 3-way merge, it means your `meld diff` window will suddenly have three panes which doesn't make any sense.

▲ 17 Reply ...



agfe2 Over a year ago

Why not check configuration, after set with `$git config -l`

▲ 0 Reply ...



89



For Windows. Run these commands in Git Bash:

```
git config --global diff.tool meld
git config --global difftool.meld.path "C:\Program Files\Meld\Meld.exe"
git config --global difftool.prompt false

git config --global merge.tool meld
git config --global mergetool.meld.path "C:\Program Files\Meld\Meld.exe"
git config --global mergetool.prompt false
```

(Update the file path for Meld.exe if yours is different.)

For Linux. Run these commands in Git Bash:

```
git config --global diff.tool meld
git config --global difftool.meld.path "/usr/bin/meld"
git config --global difftool.prompt false

git config --global merge.tool meld
git config --global mergetool.meld.path "/usr/bin/meld"
git config --global mergetool.prompt false
```

You can verify Meld's path using this command:

```
which meld
```

Share Improve this answer Follow

edited Feb 19, 2024 at 22:02

answered Apr 5, 2017 at 17:55



MarredCheese

21.4k 12 109 106

7 Comments ^

Add a comment



Allen Vork Over a year ago

I got an error when I run `git difftool` "The diff tool meld is not available as 'D:\software\melddiff\Meld.exe'"

▲ 1 Reply ...



MarredCheese Over a year ago

@AllenVork: Have you confirmed that Meld.exe is in the folder you specified? Can you run it outside of Git successfully? What does Git return when you run `git config --global --get-regexp diff*` ?

▲ 0 Reply ...



Allen Vork Over a year ago

I solved it. I change it to "D:/software/melddiff/Meld.exe" and it works. The .gitconfig's format is ubuntu not windows.

▲ 0 Reply ...



Andrei Over a year ago

I am getting the following error: `$ git difftool /mingw64/libexec/git-core/git-mergetool--lib: line 124: meld: command not found fatal: external diff died, stopping at src/main/java/com/MyController.java`

▲ 1 Reply ...



hfunes.com Over a year ago

@Andrei I had the same problem and solved it adding the complete path in the line **`cmd = meld "$LOCAL" "$REMOTE"`**. That's how I have the line now **`cmd = 'C:/Program Files (x86)/Meld/Meld.exe' "$LOCAL" "$REMOTE"`**

▲ 1 Reply ...



I prefer to set up Meld as a separate command, like so:

43

```
git config --global alias.meld '!git difftool -t meld --dir-diff'
```



This makes it similar to the *git-meld.pl* script here: [git meld README](#)



You can then just run



```
git meld
```

Share Improve this answer Follow

edited Mar 5, 2024 at 7:45

answered Feb 19, 2017 at 11:08



Peter Mortensen

30.8k 22 112 135



Ulf Adams

1,339 10 12

1 Comment ^

Add a comment



PfunnyGuy Over a year ago

I just had meld working with Cygwin, and now it's broke. This fixed it. Thanks! (Though I removed the `--dir-diff` portion as personal preference.)



For Windows 10 I had to put this in my .gitconfig:

7



```
[merge]
  tool = meld
[mergetool "meld"]
  cmd = 'C:/Program Files (x86)/Meld/Meld.exe' $LOCAL $BASE $REMOTE --output=$MERGED
[mergetool]
  prompt = false
```



Everything else you need to know is written in this awesome [answer by mattst](#).

PS: For some reason, this only worked with Meld 3.18.x, while some of the Meld 3.20.x versions give me an error. It should work with Meld 3.20.4 as mentioned by TomasMolina in a comment below.

Share Improve this answer Follow

edited Mar 5, 2024 at 7:47

answered Feb 6, 2019 at 13:36



Peter Mortensen

30.8k 22 112 135



dannybucks

2,385 2 22 28

1 Comment ^

Add a comment



TomasMolina Over a year ago

I have used this snippet and works seamlessly on Windows 10 and Meld 3.20.4



5



I follow this simple setup with Meld. Meld is a free and opensource diff tool. You will see a nice side by side comparison of files and directory for any code changes.

1. Install Meld in your Linux using [YUM](#) or [APT](#).
2. Add the following line in your `~/.gitconfig` file

```
[diff]
  tool = meld
```



3. Go to your code repository and type the following command to see the difference between the last committed changes and current working directory (unstaged uncommitted changes)

```
git difftool --dir-diff ./
```

4. To see the difference between the last committed code and staged code, use the following command

```
git difftool --cached --dir-diff ./
```

Share Improve this answer Follow

edited Mar 5, 2024 at 7:52



Peter Mortensen

30.8k 22 112 135

answered Dec 10, 2020 at 7:42



Sunil S

1,026 1 11 25

Comments

Add a comment



1



This is an answer targeting primarily developers using Windows, as the path syntax of the diff tool differs from other platforms.

I use Kdiff3 as the Git mergetool, but to set up the Git difftool as Meld, I first installed the latest version of Meld from [Meldmerge.org](https://meldmerge.org). Then I added the following to my global `.gitconfig` file using:

```
git config --global -e
```

Note, if you rather want [Sublime Text](https://www.sublimetext.com/3) 3 instead of the default Vim as the core editor, you can add this to the `.gitconfig` file:

```
[core]
editor = 'c:/Program Files/Sublime Text 3/sublime_text.exe'
```

Then you add in Meld as the difftool:

```
[diff]
tool = meld
guitool = meld

[difftool "meld"]
cmd = \"C:/Program Files (x86)/Meld/Meld.exe\" \"$LOCAL\" \"$REMOTE\" --label \"DIFF (ORIGINAL MY)\"
prompt = false
path = C:\\Program Files (x86)\\Meld\\Meld.exe
```

Note the leading slash in the cmd above. On Windows it is necessary.

It is also possible to set up an alias to show the current Git diff with a `--dir-diff` option. This will list the changed files inside Meld, which is handy when you have altered multiple files (a very common scenario indeed).

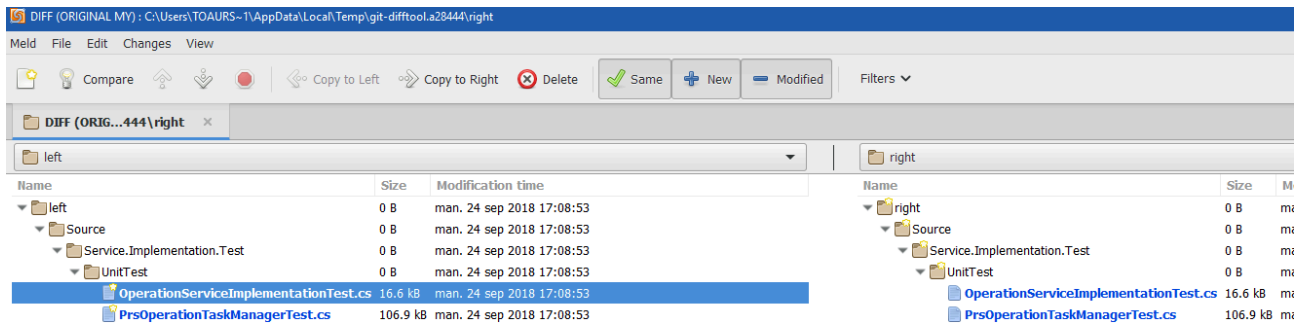
The alias looks like this inside the `.gitconfig` file, beneath the `[alias]` section:

```
showchanges = difftool --dir-diff
```

To show the changes I have made to the code, I then just enter the following command:

```
git showchanges
```

The following image shows how this `--dir-diff` option can show a listing of changed files (example):



Then it is possible to click on each file and show the changes inside Meld.

Share Improve this answer Follow

edited Mar 5, 2024 at 8:26

answered Sep 24, 2018 at 15:14



Peter Mortensen

30.8k 22 112 135



Tore Aurstad

4,083 1 36 28

Comments

Add a comment



1



It can be complicated to compute a diff in your head from the different sections in `$MERGED` and apply that. In my setup, Meld helps by showing you these diffs visually, using:

Copy

```
[merge]
tool = mymeld
conflictstyle = diff3
```

```
[mergetool "mymeld"]
cmd = meld --diff $BASE $REMOTE --diff $REMOTE $LOCAL --diff $LOCAL $MERGED -
-output $MERGED
```

It looks strange, but offers a very convenient workflow, using three tabs:

1. in tab 1 you see (from left to right) the change that you should make in tab 2 to solve the merge conflict.

2. in the right side of tab 2 you apply the "change that you should make" and copy the entire file contents to the clipboard (using `Ctrl` + `A` and `Ctrl` + `C`).
3. in tab 3 replace the right side with the clipboard contents. If everything is correct, you will now see—from left to right—the same change as shown in tab 1 (but with different contexts). Save the changes made in this tab.

Notes:

- don't edit anything in tab 1
- don't save anything in tab 2 because that will produce annoying popups in tab 3

Share Improve this answer Follow

edited Jul 31 at 0:44



wjandrea

34.4k 10 71 108

answered Dec 1, 2016 at 22:50



mnieber

1,060 10 23

3 Comments ^

Add a comment



André Werlang Over a year ago

Is this any better than the 3-way merge (local/base/remote) in a single tab?

▲ 0

Reply



mnieber Over a year ago

@AndréWerlang The advantage of the 3-way merge in a single tab is that you only need to deal with the conflicting changes (the other changes are merged automatically). But I prefer "my" approach in cases where it's hard to understand in the 3-way merge what changed, and how to merge in a way that preserves all the changes. If at some point the 3-way merge no longer confuses me then I may switch back to it.

▲ 1

Reply



André Werlang Over a year ago

As the user mattst pointed out, you could use `$BASE` instead of `$MERGED` to begin the merge operation

▲ 0

Reply



1



How to use the shell rather than Git config to get everything in [mattst's answer](#) plus a directory diff and option choice between `$BASE` and `$MERGED` in the middle pane:

Copy

```
# Diff With Meld
function dwm { git difftool --tool=meld --no-prompt $@; }

# Dir diff With Meld (or you can just use dwm -d instead)
function ddwm { git difftool --tool=meld -d $@; }
```

```
# Merge With Meld
function mwm
{
    # Merge using Meld with $BASE in the middle pane unless --merged is
    # given, in which case $MERGED starts in the middle pane.

    local my_MPIC;    # Middle Pane Initial Contents
    if [ "$1" = --merged ]; then
        my_MPIC='$MERGED'
        shift
    else
        my_MPIC='$BASE'
    fi

    local my_cmd='meld "$LOCAL" "'$my_MPIC'" "$REMOTE" --output "$MERGED"'

    git -c mergetool.meld.cmd="$my_cmd" mergetool --tool=meld $@
}
```

Share Improve this answer Follow

edited Jul 31 at 0:46

answered Jan 26, 2024 at 11:47



wjandrea

34.4k 10 71 108



Britton Kerin

503 4 11

Comments

Add a comment



0



There isn't much to add to [mattst's excellent answer](#), except for this. For people using [Windows Subsystem for Linux \(WSL 2\)](#), you can develop in Linux, but use Meld for Windows.

```
cmd = "/mnt/c/Program Files (x86)/Meld/Meld.exe" "$LOCAL" "$REMOTE"
```

One caveat: all of my code is on a Windows drive, not the WSL 2 virtual drive, so your mileage may vary.

Share Improve this answer Follow

edited Mar 5, 2024 at 8:13

answered Jul 29, 2021 at 19:48



Peter Mortensen

30.8k 22 112 135



craigim

3,914 1 25 42

Comments

Add a comment



0

Using [Meld](#) with Git on Windows:

- Install the .msi . Unfortunately it's not digitally signed (in 2024).



- It adds itself to `PATH`, therefore you don't have to specify the full path of `c:\Program Files\Meld\Meld.exe` in the following step, which would be tricky, because of escaping.



- Use the following git configuration:



```
git config --global diff.tool meld
git config --global difftool.prompt false
git config --global merge.tool meld
git config --global mergetool.keepBackup false
```

Documentation: [diff.tool](#), [difftool.prompt](#), [merge.tool](#), [mergetool.keepBackup](#). It's not necessary to set [difftool.meld.cmd](#) or [mergetool.meld.cmd](#).

Share Improve this answer Follow

edited Aug 4, 2024 at 19:10

answered Aug 4, 2024 at 17:56



Dr. Gut

3,912 15 33

Comments

Add a comment



For Windows, combining [answer 1](#), [answer 2](#).

0



```
git config --global difftool.meld.path "%LOCALAPPDATA%\Programs\Meld\meld.exe"
git config --global difftool.meld.cmd "meld $LOCAL $REMOTE"
git config --global difftool.prompt false
```



```
git config --global mergetool.meld.path "%LOCALAPPDATA%\Programs\Meld\meld.exe"
git config --global mergetool.meld.cmd "meld $LOCAL $BASE $REMOTE --
output=$MERGED"
git config --global mergetool.prompt false
```



Share Improve this answer Follow

answered Jan 1, 2025 at 10:14



Smart Manoj

6,115 6 47 66

Comments

Add a comment



The only thing you do is:

0

```
git config --global merge.tool meld
```



Git will set up the rest, including the difftool.



You can see in the Git source code that it knows how to set up a selection of merge tools, including Meld:

[git/mergetools](#)

Share Improve this answer Follow

edited Jul 30 at 17:16

answered Jul 29 at 22:07



Peter Mortensen

30.8k

22

112

135



user240515

3,217

1

32

34

Comments

Add a comment

Start asking to get answers

Find the answer to your question by asking.

Ask question

Explore related questions

git

meld

See similar questions with these tags.